Message authentication codes

CS 161 Spring 2026 - Lecture 10



Announcements

- Attendance codes
- Raluca’s OH: Wednesdays 4-5pm (walking from class and then Soda 729)

Last time: Cryptographic hash functions

- Deterministic function from arbitrary-length input to fixed-length output
- Security properties
  - **Collision resistant:** It is hard to find $x' \neq x$ such that $H(x) = H(x')$.
  - **One way:** Given a $y \leftarrow H(\text{random})$, it is hard to find an x such that $H(x) = y$.
- If H has n-bit output, can break collision resistance in $O(2^{n/2})$ time
- The magic of collision resistant hashes:
  You can authenticate a very long message by authenticating a 256-bit string

Common examples: SHA2, SHA3 (Keccak), Blake2b, …

Message Authentication Codes (MACs)



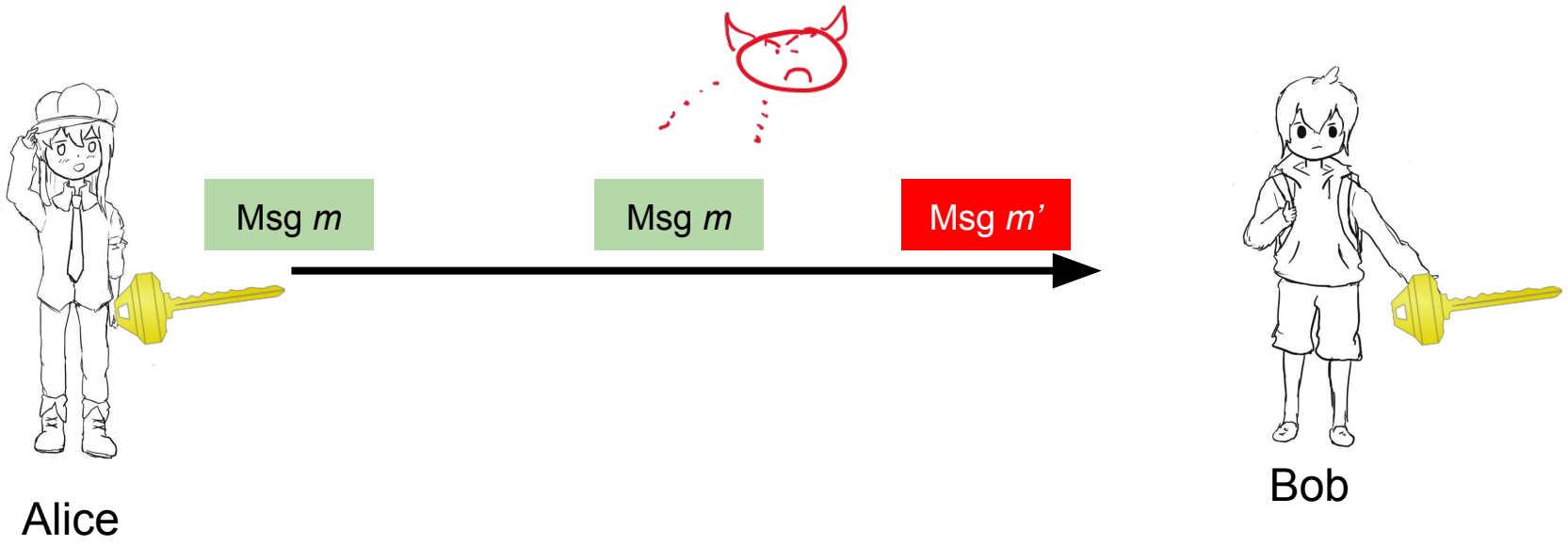
Textbook Chapter 8.1–8.3 & 8.5–8.6

Cryptography roadmap

	Secret-key crypto	**Public-key crypto**
Encryption	✅ One-time pad ✅ CPA security ✅ Counter-mode encryption ✅ CBC encryption – Authenticated encryption	– Key exchange (e.g., Diffie-Hellman) – Public-key encryption (e.g., ElGamal)
Authentication	– Message auth. codes (“MACs”)	– Digital signatures (e.g. RSA sigs)
Building blocks	✅ Pseudorandom functions ✅ Block ciphers (e.g., AES) ✅ Hash functions (e.g., SHA2)	– Discrete-log problem

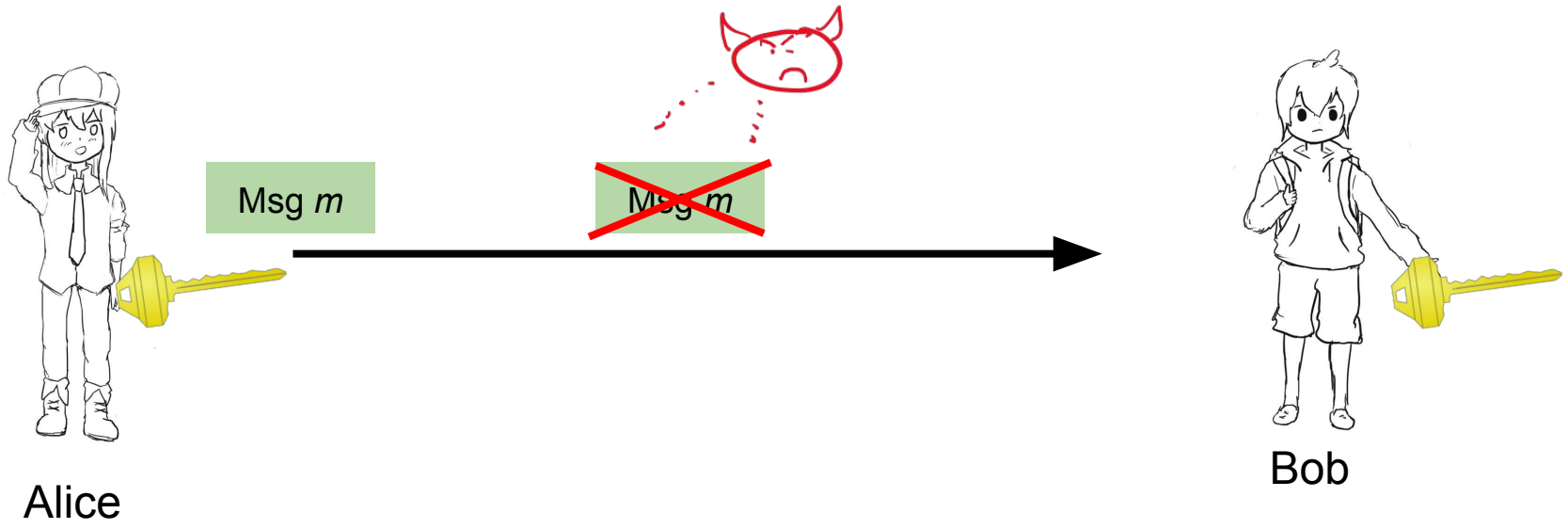
Goal: Message authentication

- **Secret-key setting:** Alice and Bob share a secret key (e.g., 128 bits)
- **Goal:** Bob should either get a message that Alice "signed," or know that he didn't



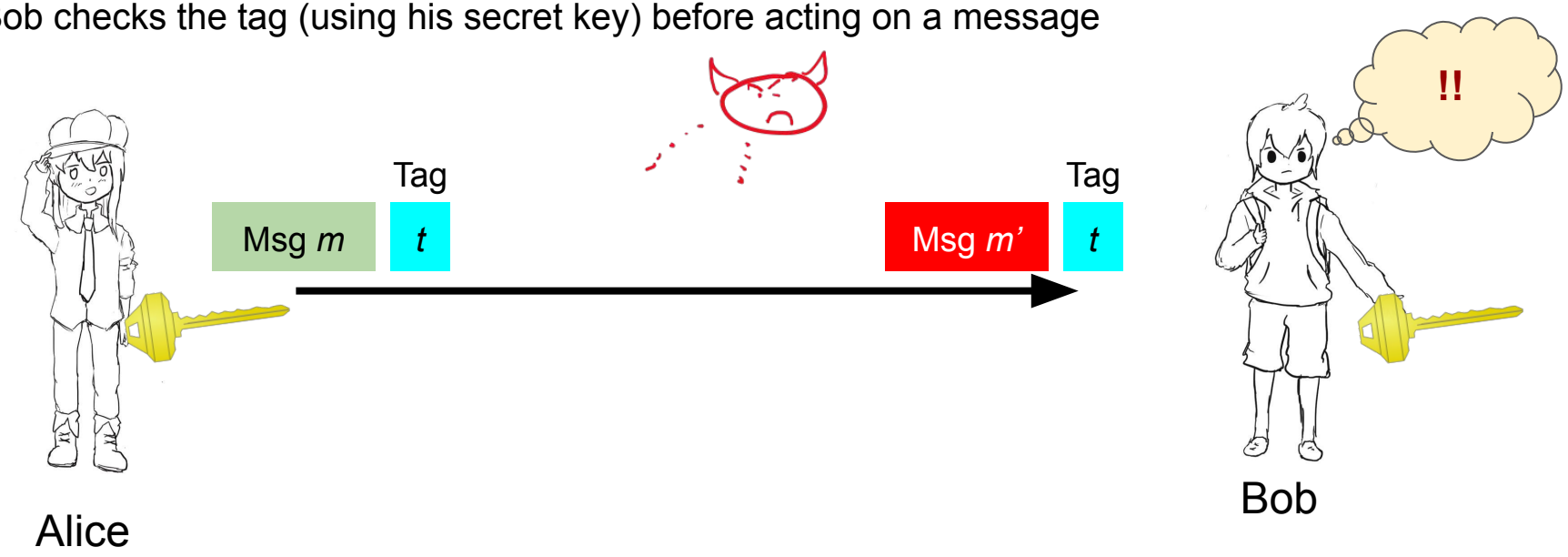
Does not protect against “denial of service”

- In-network attacker can still **block** communication
- But at least Bob will not act upon or accept a tampered message



New tool: Message authentication codes (MACs)

- Alice appends a **message-specific tag** to every message that she sends
  - Generating the tag requires Alice's secret key
- Bob checks the tag (using his secret key) before acting on a message

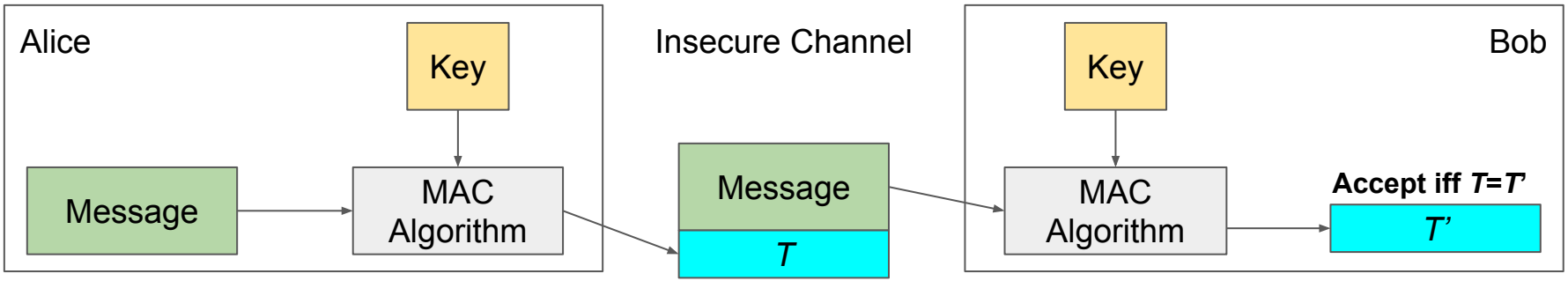


MACs: Definition

A MAC scheme consists of two efficient algorithms:

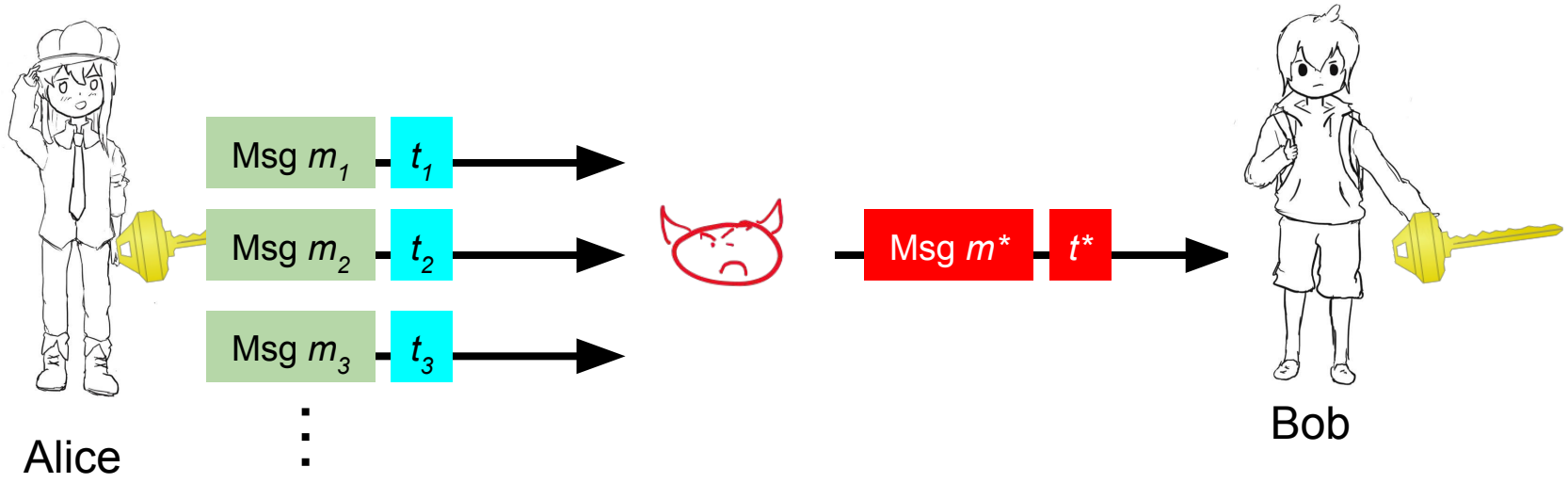
- KeyGen() → *K*. // Generate a MAC secret key – often 128-256 bits
- MAC(*K*, *M*) → *T*. // Generate a MAC tag. – often 128-256 bits

Some MAC schemes have distinct “Sign” and “Verify” algorithms

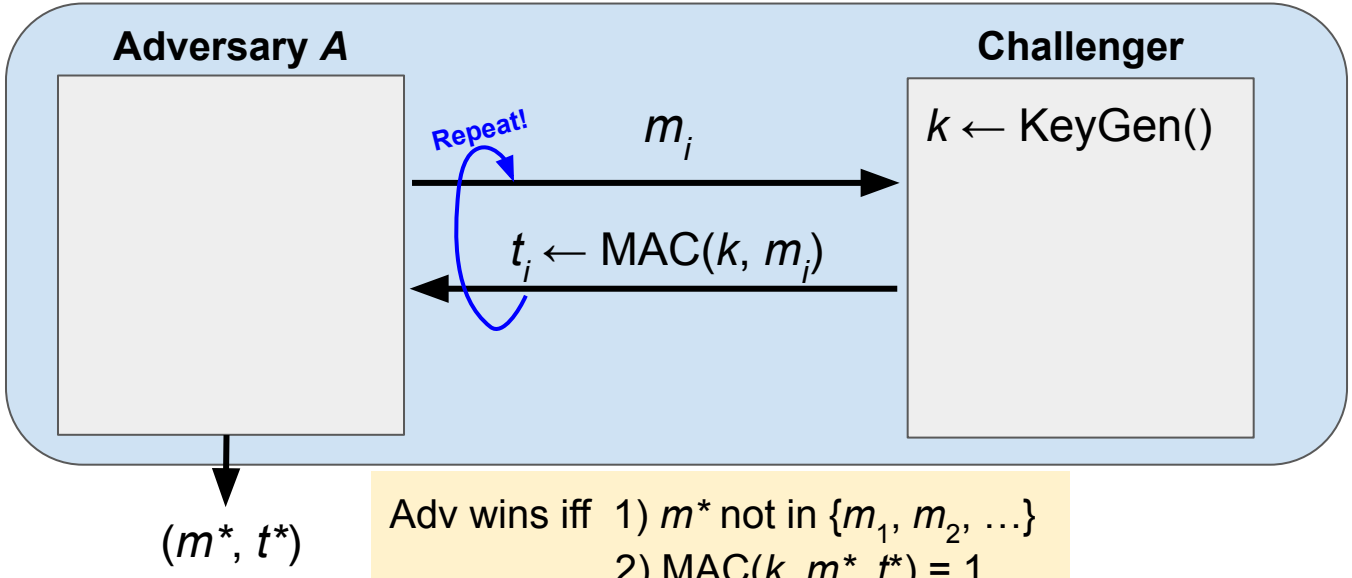


MAC Security: Basic idea

- Attacker sees many (msg, tag) pairs on messages of its choice.
- Even then, attacker cannot produce a valid tag on a single **new** message



MAC Security Game EU-CMA ("Existential unforgeability under chosen-message attack")

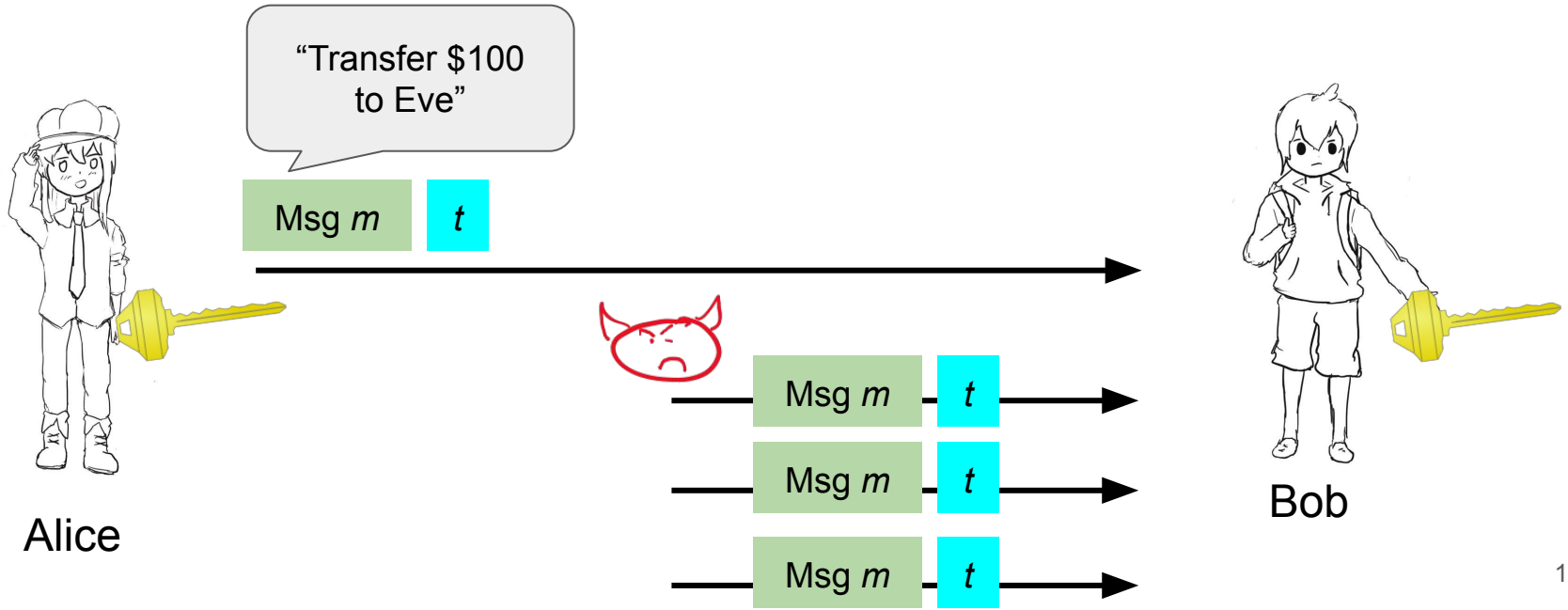


Adv wins iff 1) m^* not in $\{m_1, m_2, \ldots\}$
2) MAC(k, m^*, t^*) = 1

A MAC is *secure* if,
for all "efficient" adversaries A, Pr[A wins in MAC game] ≤ "negligible."

Warning: MAC does not protect against replay attacks!

- Attacker can always resend valid (msg, tag) pairs
- MAC security does not prevent this! Need additional defenses…



Reminder: My life advice for you

- Replay attacks are the kind of tricky thing that you don’t want to think about
  - These things are extremely subtle
  - If you are thinking about replay attacks, you are using tools that are too low-level
- Instead, use a higher-level protocol, like TLS (coming later)
  - Someone else did the hard thinking for you
- If you just need secret-key encryption (e.g., for file), use a high-level library
  - E.g., NaCl secret_box

Examples of MACs

- **HMAC-SHA2**
  HMAC-SHA3
  = Uses a cryptographic hash function to build a MAC
  // length = output length of hash function

- **CBC-MAC**
  = Uses a CBC construction to build MAC from block cipher
  // length = block size of block cipher

- **GMAC**
  = Used in the AES-GCM authenticated-encryption scheme
  (very widely used)
  // length = 128 bits

- **Poly1305**
  = A fast one-time-secure MAC secure ← **Today**
  (also widely used in ChaCha20-Poly1305 auth enc.)
  // length = 128 bits

How to build a MAC I:
Collision Resistant Hash + PRF



Textbook Chapter 8.1–8.3 & 8.5–8.6

A PRF is a secure MAC for short messages

- Say you have a PRF F with n-bit inputs and outputs
  – Or a block cipher with a n-bit block

- Here is a simple MAC construction for n-bit messages:

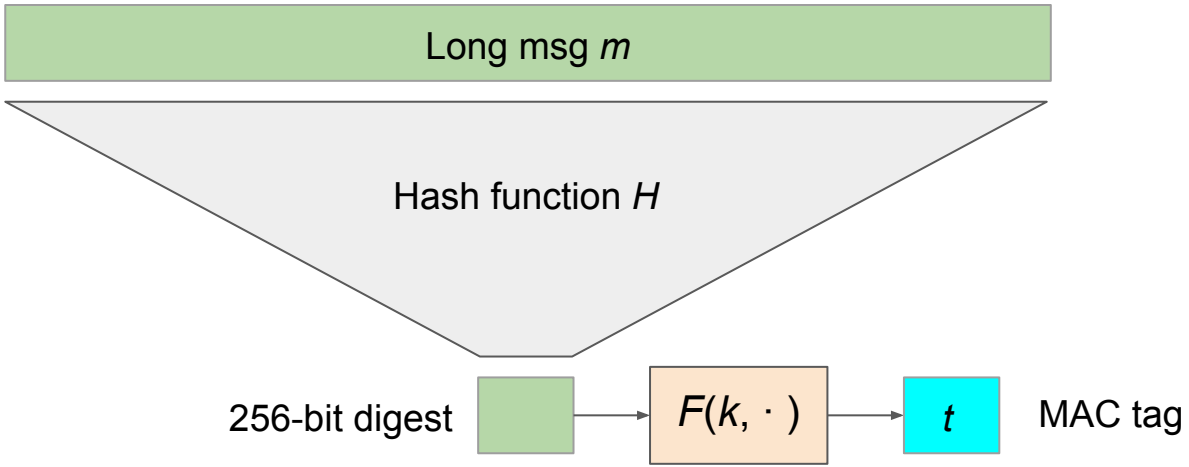
  KeyGen() → Output a key k for the PRF

  MAC(k, m) → Output $F(k, m)$

- Why secure?
  - $F(k,)$ behaves like a random function
  - For any new message m^*, $t^* \leftarrow F(k, m^*)$ is an “as good as random” value
  - Attacker has no hope of guessing a valid (m^*, t^*) pair

MAC for long messages with collision-resistant hashing

Collision-resistant hash: “Authenticate a long string by authenticating a short string”



MAC for long messages with collision-resistant hashing

- "Hash and sign" paradigm:
  - Use collision-resistant hash $H : \{0,1\}^* \rightarrow \{0,1\}^{256}$
  - Hash long message down to 256-bit digest
  - Then MAC the digest

$$\text{MAC}(k, m) := F(k, H(m))$$

- Why secure?
  - Say that attacker produces forgery (m*, t*)
  - If m^* is a new message, then $H(m^*)$ will be a different hash value than any the adversary has seen (by collision resistance)
  - Then, security of the small MAC implies that forgery will be infeasible
- Not used in practice: CRHF is slow relative to other MACs. Example next…

How to build a MAC II:
Polynomial MAC



Textbook Chapter 8.1–8.3 & 8.5–8.6

Plan

1) Build a MAC that is one-time secure

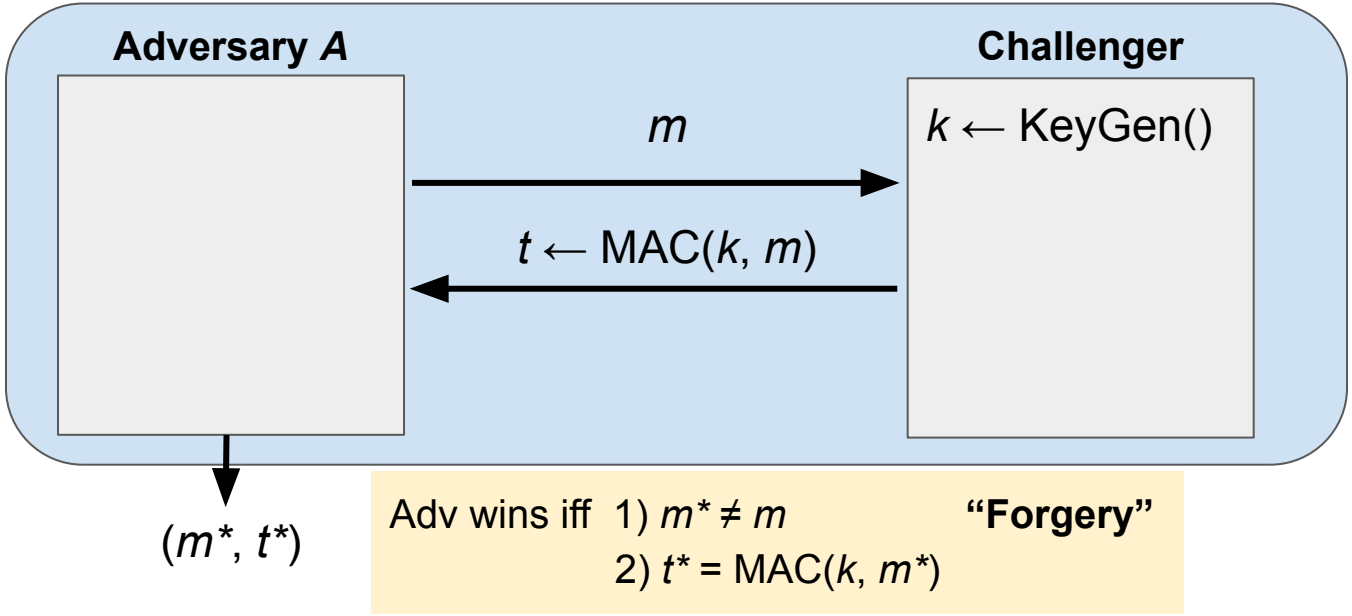
   – Can only use the secret key to MAC one message

   – Will see: Polynomial MAC

2) Use a PRF to generate **many one-time keys** from a single short secret

Popular MACs use this paradigm.

One-time MAC Security Game



A MAC is one-time *secure* if,
for all "efficient" adversaries *A*, Pr[*A* wins in MAC game] ≤ "negligible."

Construction: Polynomial MAC

Public parameters: A big integer N (> 128-bits long)

KeyGen() → Output two integers k_0, k_1 in {0, …, N - 1}

MAC(k, m) → ***t***

- Break message m into blocks of 128 bits m_1, m_2, …, m_L (with some padding)
- View each message block as an integer in {0, …, N - 1}
- Define the polynomial $M(x) = m_1x + m_2x^2 + m_3x^3 + ... + m_Lx^L \mod N$
- Output tag $t = M(k_0) + k_1$ // One-time pad encryption of M(r) with key s.

Why is polynomial MAC one-time secure?

- Values k_0, k_1 are random and secret
- Polynomial $M(x) = m_1x + m_2x^2 + m_3x^3 + ... + m_Lx^L \mod N$
- Tag $t \leftarrow M(k_0) + k_1$

Security idea: (Sketch)

- By one-time pad security, the tag reveals no information about k_0.
- ⇒ The adversary's forgery (m^*, t^*) is independent of k_0
- Let $M^*(x)$ be the polynomial associated with message m^*
- Pr[Adv forges] = Pr[$M^*(k_0) = t^*$: k_0 random in {0, …, N-1}]
  = Pr[$M^*(k_0) - t^* = 0$]
  ≤ L/N // Polynomial of degree ≤ L poly has at most L roots

Polynomial MAC is very slick

- No heavy “crypto” operations at all! Just ADD and MUL modulo *N*
  - With clever implementation: One ADD and one MUL per message block
- Easy to parallelize
- Information-theoretic security, like one-time pad

We now have a one-time MAC

- What good is that? We can only use the secret key ONE TIME
- We have seen this before with one-time pad
- Basic idea: Use a pseudorandom function (PRF)
  - For each message, generate a fresh ONE-TIME MAC key
  - Use each one-time MAC key only once
  - Requires a slightly different polynomial MAC
  - Don’t try this at home

MACs: Summary

- **Syntax:** MAC(key, message) → tag
- MAC Security
  - Attacker can see tags on many messages of its choice
  - Still, attacker cannot produce a valid tag on a *single* new message
- MACs do not provide confidentiality, do not protect against replay attacks
- Usually, you should not use MACs directly
  - They are an important building block

Next time: Authenticated encryption, the “gold standard” for encryption.